

TURNOSRED

Backend de Gestión de Turnos Médicos

Documento de Evidencia • Pruebas API REST • Node.js / Express / Socket.IO / Zod

Fecha de elaboración: 15 de septiembre de 2026

1. Alcance y Objetivo

El presente documento recopila la evidencia de las pruebas realizadas sobre la API REST TurnosRed (Node.js, TypeScript, Express y Socket.IO). Se documenta la ejecución de peticiones en el cliente Thunder Client (VS Code), la validación Zod con errores 400 estandarizados, las consultas con query params, la ejecución de tests automatizados en estado Pass y la simulación de la API mediante un Mock Server local (json-server).

Además se evidencia la **refactorización hacia Clean Architecture**: la lógica de los endpoints migró a **controladores asíncronos** por entidad (Turnos y Médicos) y a un **controller general** que gestiona el endpoint de bienvenida (**GET /**) y el manejador de **rutas inexistentes (404)**.

Tecnologías: Express 5 • Socket.IO 4 • Zod • TypeScript 6 • Postman / Newman.

2. Configuración previa para las evidencias

2.1 Servidor local

- Ejecutar el backend: `npm run dev` (puerto 3000).
- Verificar en consola: "Servidor de TurnosRed ejecutándose en el puerto 3000".

2.2 Cliente de pruebas (Thunder Client)

- Abrir la extensión **Thunder Client** en VS Code (icono de rayo en la barra lateral).
- Crear una colección "TurnosRed" y usar la base `http://localhost:3000`.
- Los scripts de test automatizados se ejecutan con **Newman** sobre la colección `postman/TurnosRed.postman_collection.json`.

2.3 Preparar el Mock Server (json-server)

- Levantar el mock local con: `npx json-server postman/mock-db.json --port 3001`
- Los datos ya están normalizados en `postman/mock-db.json` (formato objeto con rutas /turnos y /medicos).

3. Evidencia de peticiones y respuestas

Se muestran las respuestas reales obtenidas (formato JSON) mediante Thunder Client.

3.1 GET /turnos — Listar turnos

- Petición: `GET {{baseUrl}}/turnos` → Estado: **200 OK**.

```
[{"id":103,"paciente":"Ana Gomez","documento":"28999111","especialidad":"Clínica Médica","fecha":"15/08/2026","hora":"11:30","confirmado":false,"medicoId":202}, {"id":5801,"paciente":"Lucía Fernández","documento":"AB-4567 / 2026","especialidad":"Nutrición","fecha":"16/08/2026","hora":"9:30","confirmado":true,"medicoId":201}]
```

3.2 GET /medicos — Listar médicos

- Petición: **GET {{baseUrl}}/medicos** → Estado: **200 OK**.

```
[{"id":201,"nombre":"Dra. Laura Fernandez","matricula":"MN 12345","especialidad":"Pediatría","activo":true}, {"id":202,"nombre":"Dr. Martin Lopez","matricula":"67890","especialidad":"Clínica Médica","activo":true}]
```

3.3 POST /turnos — Crear turno

- Petición: **POST {{baseUrl}}/turnos** con body JSON → **201 Created**.

```
{ "id": 5801, "paciente": "Lucía Fernández", "documento": "AB-4567 / 2026", "especialidad": "Nutrición", "fecha": "16/08/2026", "hora": "9:30", "confirmado": true, "medicoId": 201 }
```

3.4 PUT /medicos/:id — Actualizar médico

- Petición: **PUT {{baseUrl}}/medicos/201** con body → **200 OK**.

```
{ "id": 201, "nombre": "Dra. Laura Fernandez", "matricula": "MN 12345", "especialidad": "Pediatría", "activo": true }
```

3.5 DELETE /turnos/:id — Eliminar turno

- Petición: **DELETE {{baseUrl}}/turnos/{id}** → **204 No Content** (sin cuerpo).

(cuerpo vacío — HTTP 204)

4. Evidencia del Controller General (bienvenida y rutas 404)

El **general.controller.ts** expone dos funciones asíncronas: **helloWorld** (endpoint **GET /**) y **notFoundRoute** (middleware para rutas inexistentes). Ambos gestionan su código de estado mediante la variable `status` y retornan con `return` `res.status(status).json(...)`.

4.1 Endpoint de bienvenida (GET /)

- GET / → **200 OK** con mensaje de bienvenida y listado de endpoints.

```
{ "message": "¡Bienvenido a la API TurnosRed!", "endpoints": [ "GET /", "GET /turnos", "GET /turnos/:id", "POST /turnos", "PUT /turnos/:id", "DELETE /turnos/:id", "GET /medicos", "GET /medicos/:id", "POST /medicos", "PUT /medicos/:id", "DELETE /medicos/:id" ], "timestamp": "2026-09-15T03:25:36.043Z" }
```

4.2 Ruta no encontrada a través del controller (404)

- GET /ruta-inexistente → **404** gestionado por **notFoundRoute** (general.controller.ts) con formato estándar.

```
{ "status": 404, "message": "Ruta no encontrada: GET /ruta-inexistente", "code": "NOT_FOUND", "details": [] }
```

4.3 Validación de ID inválido (400 INVALID_ID)

- GET /turnos/abc → **400** con code **INVALID_ID** (validación previa en el controlador antes de consultar datos).

```
{ "status": 400, "message": "El ID de turno debe ser un número entero positivo.", "code": "INVALID_ID", "details": [ { "path": "id", "message": "El ID de turno debe ser un número entero positivo." } ] }
```

5. Evidencia de validaciones Zod (error 400 estandarizado)

Se enviaron cuerpos inválidos para verificar el formato unificado de error de la API: `{ status, message, code, details }`.

5.1 Especialidad no válida (Title Case)

- POST /turnos con especialidad: "PEDIATRÍA" → **400 VALIDATION_ERROR**.

```
{ "status": 400, "message": "Error de validación en los datos ingresados.", "code": "VALIDATION_ERROR", "details": [ { "path": "especialidad", "message": "La especialidad debe estar en formato Title Case (ej. \"Clínica médica\", \"Pediatría\")." } ] }
```

5.2 Campos faltantes o tipo incorrecto

- POST /medicos con datos inválidos → **400 VALIDATION_ERROR** con details.

```
{ "status": 400, "message": "Error de validación en los datos ingresados.", "code": "VALIDATION_ERROR", "details": [] }
```

5.3 Recurso inexistente (404 NOT_FOUND)

- GET /turnos/99999 → **404** con code **NOT_FOUND** y formato estándar.

```
{ "status": 404, "message": "Turno no encontrado.", "code": "NOT_FOUND", "details": [] }
```


POST	/medicos	201	4	PASS
POST	/medicos (inválido)	400	4	PASS
PUT	/medicos/:id	200	3	PASS
DELETE	/medicos/:id	204	2	PASS

7.1 Ejecución de los tests

- Los tests automatizados se ejecutan con **Newman** sobre la colección de la API.

7.2 Newman (terminal)

- Comando: `npx newman run postman/TurnosRed.postman_collection.json -e postman/TurnosRed.postman_environment.json`

8. Evidencia de Mock Server

Se simuló la API localmente con **json-server** (los datos están normalizados en el archivo **postman/mock-db.json** con formato de objeto, donde cada clave define un endpoint). Así puede responderse la API sin necesidad del backend real.

8.1 Levantar el Mock Server

```
npx json-server postman/mock-db.json --port 3001
```

- Responderá en **http://localhost:3001** con las rutas **/turnos** y **/medicos**.
- Truco: funciona con un archivo de objeto { "turnos": [...], "medicos": [...] } (un archivo con array suelto genera 404 en json-server v1).

8.2 Petición contra el Mock (GET /turnos)

- Petición: **GET http://localhost:3001/turnos** en Thunder Client → respuesta simulada **200**.

8.3 Otras peticiones contra el Mock

- GET **http://localhost:3001/medicos** y GET **http://localhost:3001/turnos/99999** (404 simulado).
- POST /turnos (201), PUT /medicos/:id (200) y DELETE /turnos/:id (200).

9. Conclusión

La API de TurnosRed cumple con los requisitos RESTful solicitados (verbos HTTP correctos y códigos 200/201/204/400/404/500), expone errores bajo un formato estándar unificado, valida los datos con Zod (incluyendo especialidades en Title Case y documento como string), soporta filtros por query params en la capa de servicios y queda organizada bajo **Clean Architecture**: controladores asíncronos por entidad, controller general (bienvenida y rutas 404), validaciones previas con retorno anticipado y gestión dinámica de códigos de estado. Cuenta con tests automatizados en verde (sin regresiones tras la refactorización) y puede simularse localmente mediante un Mock Server de json-server.